GPUMODE Lecture Study Notes: Lecture 43
INT8 Tensor Core Matmul

Core Problem the Lecture Addresses

This lecture studies how to implement an educational but increasingly optimized **INT8 tensor core matrix multiplication** in CUDA C++ for NVIDIA Turing-class GPUs. The goal is not to beat production libraries such as cuBLAS or CUTLASS. Instead, the goal is to expose the basic mechanics of tensor cores, memory movement, profiling, register reuse, shared memory reuse, and layout transformations.

The target computation is an integer GEMM:

$$C = AB,$$

where

$$A \in \mathbb{Z}_8^{N \times K}, \qquad B \in \mathbb{Z}_8^{K \times N}, \qquad C \in \mathbb{Z}_{32}^{N \times N}.$$

The input elements are 8-bit integers, but the output accumulators are 32-bit integers because multiplying two 8-bit integers produces a wider intermediate value, and summing over the reduction dimension K requires enough dynamic range.

The central engineering question is:

> How do we feed INT8 tensor cores with the right data, in the right layout, at the right time, while avoiding excessive global memory traffic, shared memory bottlenecks, register spills, and synchronization stalls?

The lecture proceeds through a sequence of kernels:

1. A direct tensor core kernel using `nvcuda::wmma` fragments.
2. A version that stores the second matrix in column-major layout to improve memory coalescing.
3. A register-reuse version where each warp computes multiple output fragments.
4. A shared-memory tiled version where blocks cooperate to stage tiles.

5. A layout-preprocessed version that avoids repeatedly performing expensive fragment memory-to-register layout conversions.

The educational theme is that tensor cores do not automatically make code fast. The tensor core instruction may perform the arithmetic quickly, but the kernel can still be dominated by memory access patterns, instruction overhead, shared memory throughput, register pressure, synchronization, or poor scheduling.

Required Background

Matrix Multiplication

For matrices

$$A \in \mathbb{R}^{M \times K}, \qquad B \in \mathbb{R}^{K \times N},$$

the product

$$C = AB$$

has entries

$$C_{ij} = \sum_{k=0}^{K-1} A_{ik} B_{kj}.$$

The scalar algorithm performs MNK multiply-add operations. For square matrices of dimension N, the asymptotic arithmetic cost is

$$\Theta(N^3).$$

A scalar CPU-style view focuses on one output element C_{ij}. A GPU tensor core view instead focuses on small matrix fragments. For INT8 WMMA on Turing, a convenient educational shape is a $16 \times 16 \times 16$ matrix multiply:

$$C_{\text{frag}} \leftarrow A_{\text{frag}} B_{\text{frag}} + C_{\text{frag}},$$

where

$$A_{\text{frag}} \in \mathbb{Z}_8^{16 \times 16}, \qquad B_{\text{frag}} \in \mathbb{Z}_8^{16 \times 16}, \qquad C_{\text{frag}} \in \mathbb{Z}_{32}^{16 \times 16}.$$

INT8 Arithmetic and Accumulation

Integer matrix multiplication differs from floating-point matrix multiplication because the product of two 8-bit integers is not naturally an 8-bit integer. For example, multiplying two signed 8-bit values can produce a result requiring at least 16 bits before accumulation:

$$\text{int8} \times \text{int8} \rightarrow \text{int16}.$$

Accumulating many such products requires a larger accumulator:

$$\sum_{k=0}^{K-1} \text{int8} \times \text{int8} \rightarrow \text{int32}.$$

This is why INT8 GEMM commonly uses 32-bit integer accumulators. The lecture also emphasizes that integer matrix multiplication is deterministic. Unlike floating-point arithmetic, reordering integer additions does not introduce rounding error, provided the accumulator does not overflow. This makes correctness testing simpler because the reference result can be compared exactly.

GPU Execution Model

A CUDA kernel is launched over a grid of thread blocks. Each thread block runs on a streaming multiprocessor, abbreviated SM. Threads inside a block are grouped into warps of 32 threads. A warp is the basic scheduling unit for most CUDA execution.

Important terms:

- **Thread**: one scalar CUDA execution context.
- **Warp**: a group of 32 threads that usually execute instructions in lockstep.
- **Thread block**: a group of threads that can cooperate through shared memory and synchronization.
- **Grid**: all blocks launched by a kernel.
- **SM**: hardware unit containing warp schedulers, registers, shared memory, L1 cache, tensor cores, and CUDA cores.
- **Occupancy**: how many warps or blocks can be resident on an SM at the same time.
- **Tensor core**: specialized matrix-multiply hardware that performs small dense matrix operations at high throughput.

Memory Hierarchy

The relevant memory hierarchy is:

$$\text{Registers} \rightarrow \text{Shared Memory / L1} \rightarrow \text{L2 Cache} \rightarrow \text{HBM / Device Memory}.$$

Registers are fastest and private to each thread. Shared memory is explicitly managed by the programmer and shared by threads in a block. L2 cache is shared across SMs. Device memory has the largest capacity but the highest latency.

The lecture repeatedly returns to the question:

> Where is the data, and how expensive is it for the warp to get it?

Tensor Core Programming Through WMMA

CUDA exposes warp-level matrix multiply-accumulate operations through the `nvcuda::wmma` API. The important operations are:

- `wmma::fragment`: declares a matrix fragment stored across the registers of a warp.
- `wmma::fill_fragment`: initializes a fragment.
- `wmma::load_matrix_sync`: loads a matrix tile into a fragment.
- `wmma::mma_sync`: performs matrix multiply-accumulate.
- `wmma::store_matrix_sync`: stores a result fragment back to memory.

The important caveat is that the internal mapping from matrix elements to fragment registers is unspecified:

$$\text{matrix element layout} \not\equiv \text{fragment register layout}.$$

The programmer should not assume which thread owns which element of a WMMA fragment.

Main Concepts

Why INT8 Tensor Core Matmul Is Interesting

INT8 matmul is important in deep learning inference and quantized training because it reduces memory bandwidth and increases arithmetic throughput. A matrix with INT8 elements uses one fourth the storage of an FP32 matrix:

$$\frac{\text{Bytes}_{\text{int8}}}{\text{Bytes}_{\text{fp32}}} = \frac{1}{4}.$$

Lower precision can therefore improve memory traffic and allow tensor cores to execute more operations per cycle.

The lecture also frames INT8 matmul as an ideal teaching vehicle because:

- The arithmetic has exact integer correctness.
- The accumulator type difference forces the programmer to think about data types.
- Tensor core usage is visible without requiring very new hardware.
- Turing GPUs already support INT8 tensor core operations.
- The same optimization sequence mirrors scalar CUDA GEMM optimization: coalescing, register tiling, shared memory tiling, and layout transformation.

From Scalar Matmul to Fragment Matmul

The scalar decomposition is:

$$C_{ij} = \sum_{k=0}^{K-1} A_{ik} B_{kj}.$$

The tensor core decomposition groups the matrices into 16×16 fragments. Assume all dimensions are divisible by 16. Define:

$$N' = \frac{N}{16}, \qquad K' = \frac{K}{16}.$$

Then the matrix product can be written at the block-fragment level:

$$C_{IJ} = \sum_{T=0}^{K'-1} A_{IT} B_{TJ},$$

where each C_{IJ}, A_{IT}, and B_{TJ} is a 16×16 tile.

This is the same algorithmic structure as scalar GEMM, but the basic multiply operation is now a tensor core instruction instead of a scalar multiply-add.

Naive Tensor Core Kernel

A minimal WMMA kernel assigns one warp to compute one 16×16 output fragment. It repeatedly loads one fragment from A, one fragment from B, performs an MMA, and stores the output.

Conceptually:

$$C_{\text{frag}} \leftarrow \sum_{t=0}^{K/16-1} A_{\text{frag}}(t) B_{\text{frag}}(t).$$

The lecture shows that this direct approach is much slower than the speed of light. Tensor cores are used, but they are starved by poor memory access patterns and insufficient data reuse.

Memory Layout of Matrix B

If both A and B are stored row-major, then accessing rows of A is natural, but accessing columns of B during GEMM is strided. A strided access pattern across a warp can produce uncoalesced memory transactions.

For row-major storage, the address of B_{kj} is:

$$\text{addr}(B_{kj}) = \text{base}(B) + (kN + j) \cdot \text{sizeof}(B_{kj}).$$

For fixed j and changing k, consecutive values are separated by N elements. This is bad for coalescing when a warp wants a contiguous tile.

The lecture fixes this by storing or preprocessing B in column-major form. Then:

$$\text{addr}(B_{kj}) = \text{base}(B) + (jK + k) \cdot \text{sizeof}(B_{kj}),$$

so consecutive k-values become contiguous in memory.

The transpose or layout-conversion cost is only:

$$\Theta(N^2),$$

while the GEMM cost is:

$$\Theta(N^3).$$

For sufficiently large matrices, the preprocessing cost is small compared with the GEMM.

Register Reuse Through Warp Tiling

The next optimization is to make each warp compute more than one output fragment. Instead of one warp computing a single 16×16 tile, it can compute a small grid of output fragments, for example 3×3.

A 3×3 warp tile computes:

$$9 \text{ output fragments.}$$

For each reduction step, the warp needs:

$$3 \text{ fragments from } A \quad \text{and} \quad 3 \text{ fragments from } B.$$

Thus it loads 6 input fragments and performs 9 fragment MMAs.

A rough reuse ratio is:

$$\text{Reuse} = \frac{\text{output fragment updates}}{\text{input fragments loaded}} = \frac{9}{6} = 1.5.$$

Compared with a one-output-fragment warp that loads two input fragments per MMA, the register-tiled version reuses loaded fragments across multiple outputs.

The central trade-off is register pressure:

$$\text{more output fragments} \Rightarrow \text{more accumulator registers} \Rightarrow \text{lower occupancy.}$$

Shared Memory Tiling

After using registers, the next memory level to exploit is shared memory. A thread block can cooperatively load tiles of A and B from global memory into shared memory, synchronize, and then have multiple warps reuse that data.

The general structure is:

Global Memory $\rightarrow$ Shared Memory $\rightarrow$ WMMA Fragments in Registers $\rightarrow$ Tensor Cores.

This reduces global memory traffic because a tile loaded once into shared memory can be reused by several warps in the same thread block.

However, shared memory is not infinitely fast. The lecture shows that after global memory bottlenecks are reduced, shared memory-related stalls become visible, including `stall_mio_throttle` and `stall_short_scoreboard`.

Pre-Shuffling and Fragment Layout

The WMMA API performs a hidden mapping from memory layout to fragment register layout during `load_matrix_sync`. This mapping is unspecified. The lecture proposes a clever educational optimization:

> Do not assume what the fragment layout means. Instead, assume that the layout is stable during one program execution, and copy the fragment's raw register storage linearly.

The idea is:

1. Use `load_matrix_sync` once to convert memory layout into fragment layout.
2. Store the raw fragment storage to memory in a linear form.
3. Later reload that linear form directly into the fragment's internal storage.

This avoids repeatedly paying for the memory-to-fragment layout conversion in the inner loop. It also reduces address computation and can eliminate register spills caused by complex indexing.

Hardware-Level Explanation

Tensor Cores Are Not the Whole Kernel

A tensor core instruction performs a small matrix multiply-accumulate. But the overall kernel also includes:

- loading tiles from global memory,
- computing addresses,
- moving data through L2 and L1,
- staging tiles in shared memory,
- loading shared memory into registers,
- synchronizing threads,
- storing final outputs,
- handling loop control,

- avoiding register spills.

A kernel can issue tensor core instructions and still run slowly if the tensor cores wait for data. Therefore, the relevant goal is not merely:

Use tensor cores.

The real goal is:

Keep tensor cores fed with useful work.

Warp-Level Execution of WMMA

A WMMA fragment is distributed across a warp. Each thread owns some subset of the fragment's underlying registers, but the mapping is not part of the programming contract.

For a 16×16 accumulator fragment:

$$16 \times 16 = 256$$

output elements are distributed across 32 lanes, so each thread conceptually contributes to several accumulator values. The exact ownership should not be hard-coded when using the high-level WMMA API.

Global Memory Coalescing

Coalescing means that threads in a warp access adjacent memory locations so the hardware can combine their accesses into fewer memory transactions. In the naive row-major B case, the accesses are strided. If the stride is N, then adjacent logical elements along the reduction dimension have addresses separated by:

$$N \cdot \text{sizeof(int8)}.$$

This creates inefficient memory transactions.

Changing B to column-major makes the reduction dimension contiguous:

$$1 \cdot \text{sizeof(int8)}.$$

This directly reduces global memory request count and improves effective bandwidth.

Register Reuse and Occupancy

Registers are the fastest storage available to the kernel. Reusing data in registers is usually better than sending it back to shared or global memory.

However, every additional live fragment consumes registers. If each warp holds many fragments:

$$R_{\text{thread}} = R_{\text{base}} + R_{\text{A fragments}} + R_{\text{B fragments}} + R_{\text{C fragments}},$$

then the number of resident warps may decrease:

$$\text{Occupancy} \propto \frac{R_{\text{SM}}}{R_{\text{thread}} \cdot T_{\text{block}}},$$

where R_{SM} is the register file capacity per SM and T_{block} is the number of threads per block.

The lecture notes that a 3×3 register tile was useful, while larger tiles such as 4×4 may create too much register pressure or even cause register spilling.

Register Spilling

Register spilling happens when the compiler cannot keep all live values in registers. It moves some values into local memory. Despite the name, local memory usually resides in global memory space and is cached by L1 and L2.

A spill path can be summarized as:

$$\text{Registers} \rightarrow \text{Local Memory} \rightarrow \text{L1} \rightarrow \text{L2} \rightarrow \text{Device Memory if needed.}$$

In the profiling discussion, a sudden increase in local memory traffic and L2 writes indicates register spilling. Even if the program did not explicitly use more global memory, the compiler may generate hidden local memory traffic.

Shared Memory Bottlenecks

Shared memory is faster than global memory, but not free. When many warps or lanes request shared memory at a high rate, the memory pipeline can become throttled.

Relevant stall metrics include:

- `stall_mio_throttle`: the memory input/output pipeline cannot accept more memory operations.
- `stall_short_scoreboard`: the warp issued a shared-memory request but tried to use the result before it was ready.
- `stall_long_scoreboard`: the warp is waiting on a longer-latency memory dependency, typically global memory.
- `stall_lg_throttle`: global/local memory load pipeline throttling.
- `stall_barrier`: threads are waiting at synchronization barriers.

Shared memory bank conflicts can worsen these stalls because a bank conflict increases the number of cycles needed to satisfy a shared-memory request.

Synchronization Costs

When using shared memory, threads must synchronize after cooperative loading:

$$\_\_syncthreads();$$

This ensures all writes to shared memory are visible before other warps read from the shared tile.

The cost is that faster warps may wait for slower warps. If occupancy is high, the SM can schedule other resident warps while one block waits. If occupancy is low, a barrier can become much more expensive because there may be fewer independent warps available to hide the latency.

Visual Concept

Draw an SM containing registers, shared memory, warp schedulers, CUDA cores, and tensor cores. Show arrows from global memory through L2 into shared memory, then from shared memory into registers, and finally from registers into tensor cores.

Mathematical Reasoning

Arithmetic Count

For square GEMM:

$$C = AB, \qquad A, B, C \in \mathbb{R}^{N \times N},$$

each output element performs N multiply-add steps. Counting one multiply and one add as two operations:

$$\text{Ops} = 2N^3.$$

For INT8 tensor core GEMM, the operation count is often reported in integer operations or equivalent multiply-accumulate throughput. The key scaling remains:

$$\Theta(N^3).$$

Memory Cost of Preprocessing

Transposing or pre-shuffling a matrix reads and writes each element once:

$$\text{Bytes}_{\text{preprocess}} = 2N^2 \cdot \text{sizeof(int8)}$$

for one input matrix.

The GEMM itself has arithmetic cost:

$$\Theta(N^3).$$

Therefore, the relative preprocessing cost decreases as N grows:

$$\frac{\Theta(N^2)}{\Theta(N^3)} = \Theta\left(\frac{1}{N}\right).$$

This explains why preprocessing can be worthwhile for large matrices.

Fragment-Level GEMM

Let:

$$b = 16.$$

Assume:

$$N = bN', \qquad K = bK'.$$

Define matrix tiles:

$$A_{IT} \in \mathbb{Z}_8^{b\times b}, \qquad B_{TJ} \in \mathbb{Z}_8^{b\times b}, \qquad C_{IJ} \in \mathbb{Z}_{32}^{b\times b}.$$

Then:

$$C_{IJ} = \sum_{T=0}^{K'-1} A_{IT}B_{TJ}.$$

This is exactly the operation structure exposed by tensor cores:

$$D \leftarrow A_{\text{frag}}B_{\text{frag}} + C_{\text{frag}}.$$

Arithmetic Intensity

Arithmetic intensity is:

$$\text{AI} = \frac{\text{Operations}}{\text{Bytes moved}}.$$

A memory-bound kernel has low arithmetic intensity. A compute-bound kernel has high arithmetic intensity and can approach tensor core peak throughput.

For a naive fragment-level kernel, each fragment load is used for only one MMA. Register tiling increases reuse. For a warp computing an $r \times r$ tile of output fragments, each reduction step loads:

$$r \text{ fragments of } A + r \text{ fragments of } B = 2r$$

input fragments, and performs:

$$r^2$$

MMA operations. A simple reuse measure is:

$$\text{Reuse}(r) = \frac{r^2}{2r} = \frac{r}{2}.$$

For $r = 1$:

$$\text{Reuse}(1) = \frac{1}{2}.$$

For $r = 3$:

$$\text{Reuse}(3) = \frac{3}{2}.$$

Thus increasing r improves reuse, but it also increases accumulator storage:

$$\text{Accumulator fragments} = r^2.$$

Register Pressure Model

Let:

$$R_C = \text{registers per accumulator fragment per thread},$$

$$R_A = \text{registers per } A \text{ fragment per thread},$$

$$R_B = \text{registers per } B \text{ fragment per thread}.$$

For an $r \times r$ warp tile, a rough register model is:

$$R_{\text{thread}}(r) = R_{\text{base}} + r^2 R_C + r R_A + r R_B.$$

The dominant term is usually:

$$r^2 R_C.$$

This explains why increasing from 3×3 to 4×4 can be much more expensive than it first appears:

$$3^2 = 9, \qquad 4^2 = 16.$$

That is a 77.8% increase in accumulator fragments:

$$\frac{16 - 9}{9} = \frac{7}{9} \approx 0.778.$$

Tail Effect

If the number of thread blocks is not well matched to the number of SMs, some SMs may finish earlier while a small number of straggler blocks continue to run. If:

$$B_{\text{total}} \not\equiv 0 \pmod{S_{\text{SM}}},$$

where B_{total} is the number of blocks and S_{SM} is the number of SMs, the final scheduling wave may underutilize the GPU. Nsight Compute may report this as a tail effect.

Code Walkthrough

Scalar Matmul Skeleton

The scalar version computes one output element per logical worker:

```
int32_t acc = 0;

for (int k = 0; k < K; ++k) {
    int8_t a = A[i * K + k];
    int8_t b = B[k * N + j];
    acc += static_cast<int32_t>(a) * static_cast<int32_t>(b);
}

C[i * N + j] = acc;
```

Each iteration loads one element of A, one element of B, multiplies them, and accumulates into a 32-bit result. This code is easy to understand but does not use tensor cores and has poor data reuse if implemented naively.

Minimal WMMA INT8 Tensor Core Kernel

A simplified WMMA version looks like this:

```
#include <mma.h>
using namespace nvcuda;

__global__ void int8_wmma_kernel(
    const int8_t* A,
    const int8_t* B,
    int32_t* C,
    int N,
    int K) {

    int warp_id = (blockIdx.x * blockDim.x + threadIdx.x) / 32;
    int lane_id = threadIdx.x % 32;

    int tiles_per_row = N / 16;
    int tile_i = warp_id / tiles_per_row;
    int tile_j = warp_id % tiles_per_row;

    wmma::fragment<wmma::matrix_a, 16, 16, 16,
                   signed char, wmma::row_major> a_frag;

    wmma::fragment<wmma::matrix_b, 16, 16, 16,
                   signed char, wmma::row_major> b_frag;

    wmma::fragment<wmma::accumulator, 16, 16, 16,
                   int> c_frag;

    wmma::fill_fragment(c_frag, 0);

    for (int tile_k = 0; tile_k < K; tile_k += 16) {
```

```
        const int8_t* a_tile = A + tile_i * 16 * K + tile_k;
        const int8_t* b_tile = B + tile_k * N + tile_j * 16;

        wmma::load_matrix_sync(a_frag, a_tile, K);
        wmma::load_matrix_sync(b_frag, b_tile, N);

        wmma::mma_sync(c_frag, a_frag, b_frag, c_frag);
    }

    int32_t* c_tile = C + tile_i * 16 * N + tile_j * 16;
    wmma::store_matrix_sync(c_tile, c_frag, N, wmma::mem_row_major);
}
```

Important details:

- The warp is the unit of WMMA execution.
- The accumulator fragment is initialized to zero.
- The loop over K advances by 16 because each tensor core operation consumes a $16 \times 16 \times 16$ tile.
- The leading dimension passed to `load_matrix_sync` is the stride between consecutive rows or columns, depending on the layout.
- The code assumes all dimensions are multiples of 16.
- The shown indexing is pedagogical; production code must handle grid bounds, multiple warps per block, and edge cases.

This kernel uses tensor cores but still performs poorly because the memory layout of B causes inefficient loads.

Using Column-Major B

If B is stored in column-major layout, the fragment declaration and indexing change:

```
wmma::fragment<wmma::matrix_b, 16, 16, 16,
               signed char, wmma::col_major> b_frag;

const int8_t* b_tile = B_col_major + tile_j * 16 * K + tile_k;

wmma::load_matrix_sync(b_frag, b_tile, K);
```

This makes the reduction dimension contiguous for B. The extra transpose or layout conversion is a separate $O(N^2)$ preprocessing step, which is usually cheap for large GEMMs compared with the $O(N^3)$ matrix multiply.

Register-Tiled Warp Kernel

A warp can compute a 3×3 tile of output fragments:

```
constexpr int WT = 3;

wmma::fragment<wmma::accumulator, 16, 16, 16, int>
    c_frag[WT][WT];

for (int ii = 0; ii < WT; ++ii) {
    for (int jj = 0; jj < WT; ++jj) {
        wmma::fill_fragment(c_frag[ii][jj], 0);
    }
}

for (int tile_k = 0; tile_k < K; tile_k += 16) {
    wmma::fragment<wmma::matrix_a, 16, 16, 16,
                   signed char, wmma::row_major> a_frag[WT];

    wmma::fragment<wmma::matrix_b, 16, 16, 16,
                   signed char, wmma::col_major> b_frag[WT];

    for (int ii = 0; ii < WT; ++ii) {
        const int8_t* a_tile =
            A + (base_i + ii * 16) * K + tile_k;
        wmma::load_matrix_sync(a_frag[ii], a_tile, K);
    }

    for (int jj = 0; jj < WT; ++jj) {
        const int8_t* b_tile =
            B_col + (base_j + jj * 16) * K + tile_k;
        wmma::load_matrix_sync(b_frag[jj], b_tile, K);
    }

    for (int ii = 0; ii < WT; ++ii) {
        for (int jj = 0; jj < WT; ++jj) {
            wmma::mma_sync(
                c_frag[ii][jj],
                a_frag[ii],
                b_frag[jj],
                c_frag[ii][jj]);
        }
    }
}
```

This code improves reuse because each loaded A fragment is used with several B fragments, and each loaded B fragment is used with several A fragments.

The hardware implication is that more data remains in registers across multiple MMA instructions.

The cost is that the warp now owns nine accumulator fragments. This increases register pressure and may reduce occupancy.

Shared Memory Tiling Skeleton

A block-level shared-memory version stages input tiles:

```
extern __shared__ int8_t smem[];

int8_t* As = smem;
int8_t* Bs = smem + A_TILE_BYTES;

for (int tile_k = 0; tile_k < K; tile_k += BLOCK_K) {

    int linear_thread = threadIdx.x;
    int elements_per_vector = 16;

    int global_offset_a = compute_global_a_offset(
        blockIdx.y, tile_k, linear_thread);

    int shared_offset_a = compute_shared_a_offset(linear_thread);

    *reinterpret_cast<int4*>(&As[shared_offset_a]) =
        *reinterpret_cast<const int4*>(&A[global_offset_a]);

    int global_offset_b = compute_global_b_offset(
        blockIdx.x, tile_k, linear_thread);

    int shared_offset_b = compute_shared_b_offset(linear_thread);

    *reinterpret_cast<int4*>(&Bs[shared_offset_b]) =
        *reinterpret_cast<const int4*>(&B[global_offset_b]);

    __syncthreads();

    for (int inner_k = 0; inner_k < BLOCK_K; inner_k += 16) {
        wmma::load_matrix_sync(a_frag, As + a_smem_offset, 16);
        wmma::load_matrix_sync(b_frag, Bs + b_smem_offset, 16);
        wmma::mma_sync(c_frag, a_frag, b_frag, c_frag);
    }

    __syncthreads();
}
```

The use of `int4` is a vectorized-load trick. An `int4` is 16 bytes, so reinterpreting aligned INT8 memory as `int4` tells the compiler that it can generate wider memory operations.

This requires correct alignment:

$$\text{address} \equiv 0 \pmod{16}.$$

If the address is not 16-byte aligned, the reinterpret cast can be unsafe or inefficient.

The two `__syncthreads()` calls are necessary because all threads in the block cooperate to populate shared memory, and no warp should read a shared tile before it is fully written.

Direct Fragment Store and Load Helpers

The lecture proposes helper functions that copy raw fragment storage without interpreting the matrix-element mapping:

```
template <typename Frag>
__device__ void store_direct(uint32_t* dst, const Frag& frag) {
    int lane = threadIdx.x % 32;

    #pragma unroll
    for (int i = 0; i < Frag::num_elements; ++i) {
        dst[lane * Frag::num_elements + i] =
            reinterpret_cast<const uint32_t*>(frag.x)[i];
    }

    __syncwarp();
}

template <typename Frag>
__device__ void load_direct(Frag& frag, const uint32_t* src) {
    int lane = threadIdx.x % 32;

    #pragma unroll
    for (int i = 0; i < Frag::num_elements; ++i) {
        reinterpret_cast<uint32_t*>(frag.x)[i] =
            src[lane * Frag::num_elements + i];
    }

    __syncwarp();
}
```

The key principle is that the code does not ask which matrix element each register represents. It merely stores and reloads the same raw register layout. This avoids depending on the semantic mapping of WMMA fragments.

A more cautious production version would need to verify type sizes, alignment, and compiler behavior. The lecture presents this as an educational optimization within the constraints of the WMMA abstraction.

Pre-Shuffling Kernel

Instead of performing the layout conversion inside every main GEMM iteration, a separate preprocessing kernel can convert the matrix once:

```
__global__ void preshuffle_a(
    const int8_t* A,
    uint32_t* A_shuffled,
    int N,
    int K) {

    int warp_id = (blockIdx.x * blockDim.x + threadIdx.x) / 32;

    int tiles_per_row = K / 16;
    int tile_i = warp_id / tiles_per_row;
    int tile_k = warp_id % tiles_per_row;

    wmma::fragment<wmma::matrix_a, 16, 16, 16,
                   signed char, wmma::row_major> a_frag;

    const int8_t* a_tile =
        A + tile_i * 16 * K + tile_k * 16;

    wmma::load_matrix_sync(a_frag, a_tile, K);

    uint32_t* out =
        A_shuffled + warp_id * 32 * decltype(a_frag)::num_elements;

    store_direct(out, a_frag);
}
```

This changes the main GEMM kernel so it can load prearranged fragment data more directly. The preprocessing cost is $O(N^2)$, but the saved work occurs inside the $O(N^3)$ GEMM loop.

Performance Intuition

Why the First Tensor Core Version Is Slow

The first tensor core version is slow because tensor core arithmetic is not the only cost. It performs many inefficient global memory loads. Matrix B is accessed with a strided pattern, causing poor memory coalescing.

Even if the arithmetic instruction is fast, the warp can stall before issuing or completing loads. This appears in profiler metrics such as:

`stall_lg_throttle` and `stall_long_scoreboard`.

Why Column-Major B Helps

Column-major B makes memory accesses more contiguous for the reduction dimension. This reduces the number of memory transactions and lowers traffic between device memory, L2, and L1.

The important lesson is:

> A mathematically identical GEMM can have radically different performance depending on physical memory layout.

Why Register Reuse Helps

Register reuse helps because registers are the fastest storage. If a warp loads an A fragment once and uses it with multiple B fragments, the cost of that load is amortized across several MMA instructions.

For a 3×3 warp tile:

$$6 \text{ input fragments} \rightarrow 9 \text{ MMA operations.}$$

This reduces memory pressure and address-computation overhead.

The drawback is:

$$\text{more fragments} \rightarrow \text{more registers} \rightarrow \text{lower occupancy} \rightarrow \text{less latency hiding.}$$

Why Shared Memory Helps

Shared memory helps when multiple warps in a block need overlapping tiles. A global tile can be loaded once and reused by many fragment-level operations.

However, shared memory introduces:

- explicit synchronization,
- shared-memory address calculations,
- possible bank conflicts,
- shared-memory pipeline throttling,
- additional register pressure from staging logic.

Thus shared memory is a tool, not a guaranteed speedup.

Why Register Spilling Can Appear Unexpectedly

Register spilling may appear after adding shared memory even if the programmer does not explicitly allocate more global memory. The reason is that the compiler needs extra temporary registers for address calculations, loop variables, fragments, pointers, and scheduling. If the register budget is exceeded, values spill to local memory.

The profiling signature is:

$$\text{local memory traffic increases} \quad \text{and} \quad \text{L2 write traffic increases.}$$

Why Pre-Shuffling Helps

Pre-shuffling helps because the conversion from matrix memory layout to WMMA fragment register layout is no longer repeatedly done in the main loop. The kernel can reduce:

- address calculations,
- shared-memory layout overhead,
- register spills,
- instruction count,
- shared-memory load pressure.

This is not a massive early-stage speedup like fixing memory coalescing, but it becomes useful once larger bottlenecks have already been addressed.

Speed-of-Light Thinking

The lecture repeatedly compares measured performance against theoretical peak. If a GPU has peak INT8 tensor core throughput:

$$P_{\text{peak}},$$

and the kernel achieves:

$$P_{\text{measured}},$$

then utilization is:

$$U = \frac{P_{\text{measured}}}{P_{\text{peak}}}.$$

A utilization such as 40% means that the kernel still leaves substantial tensor core throughput unused. The remaining gap may come from memory stalls, instruction overhead, synchronization, insufficient pipelining, or poor scheduling.

Common Misconceptions

Misconception: Using Tensor Cores Automatically Makes GEMM Fast

Tensor cores only accelerate the MMA operation. They do not automatically solve data movement. A tensor core kernel can still be memory-bound or instruction overhead-bound.

Misconception: Shared Memory Is Always Fast Enough

Shared memory is much faster than global memory, but it can still bottleneck. High shared-memory request rates, bank conflicts, and scoreboard dependencies can cause stalls.

Misconception: Higher Occupancy Always Means Higher Performance

Occupancy helps hide latency, but high occupancy is not the goal by itself. A kernel with lower occupancy but much better register reuse can be faster. The right balance depends on the bottleneck.

Misconception: Register Spilling Only Happens When You Allocate Big Arrays

Register spilling can happen because of compiler-generated temporaries, fragment storage, address calculations, and scheduling constraints. It may appear after adding shared-memory tiling or increasing warp tile size.

Misconception: WMMA Fragments Have a Known Element Layout

The WMMA API does not guarantee the mapping from matrix elements to fragment registers. Code should not assume a specific lane owns a specific matrix element. The direct-store trick only copies raw fragment storage and reloads it consistently; it does not interpret the layout.

Misconception: Preprocessing Is Wasteful

A transpose or pre-shuffle costs $O(N^2)$, while GEMM costs $O(N^3)$. For large matrices, preprocessing can be cheap relative to the main computation and can enable much better memory behavior.

Practical Takeaways

1. Start with correctness, especially for INT8 where exact comparison is possible.
2. Use simple matrix shapes first: large, square, and divisible by 16.
3. Think in tensor core fragments, not only scalar elements.
4. Fix global memory coalescing before chasing smaller bottlenecks.
5. Store or preprocess B in a layout that makes the reduction dimension contiguous.
6. Reuse data in registers before using shared memory when possible.
7. Increase warp tile size carefully because accumulator fragments consume many registers.
8. Use shared memory to amortize global memory traffic across warps in a block.
9. Profile after each optimization. Intuition alone is not enough.
10. Watch `stall_lg_throttle`, `stall_long_scoreboard`, `stall_mio_throttle`, `stall_short_scoreboard`, and `stall_barrier`.
11. Treat local memory traffic as a warning sign for register spills.
12. Remember that preprocessing kernels may be worthwhile because GEMM is asymptotically more expensive than layout conversion.
13. Do not assume WMMA fragment internal layouts unless you are deliberately working with raw storage in a controlled way.

Project or Experiment Ideas

Experiment 1: Naive WMMA Versus Column-Major B

Implement two kernels:

1. WMMA INT8 GEMM with row-major A and row-major B.
2. WMMA INT8 GEMM with row-major A and column-major B.

Measure:

- runtime,
- achieved integer operations per second,
- global memory transactions,
- L2 traffic,

- `stall_lg_throttle`,
- `stall_long_scoreboard`.

Experiment 2: Warp Tile Size Sweep

Test warp tile sizes:

$$1 \times 1, \qquad 2 \times 2, \qquad 3 \times 3, \qquad 3 \times 4, \qquad 4 \times 4.$$

Measure runtime, registers per thread, occupancy, and spilling. The expected result is that reuse improves with tile size until register pressure dominates.

Experiment 3: Shared Memory Tiling

Add shared-memory staging. Compare against the register-only version. Profile whether global-memory stalls decrease and whether shared-memory stalls appear.

Experiment 4: Vectorized Loads

Compare scalar INT8 loads against 16-byte vectorized loads using `int4`. Verify alignment assumptions and inspect generated SASS or Nsight Compute instruction statistics.

Experiment 5: Pre-Shuffling

Implement a preprocessing kernel that converts input matrices into a raw fragment-friendly layout. Measure:

- preprocessing time,
- GEMM time,
- total time,
- instruction count,
- register spills,
- shared-memory stalls.

Experiment 6: Correctness Testing With Structured Matrices

Use structured matrices instead of only random inputs. Examples:

- identity matrix,
- block-constant matrix,
- matrices with known row or column patterns,

- small tiled matrices repeated over a large shape.

This makes indexing bugs easier to identify because wrong block boundaries become visible.

Experiment 7: Freivalds-Style Verification

Given a candidate output C, test whether:

$$Cx = A(Bx)$$

for random vectors x. This probabilistically verifies matrix multiplication without explicitly computing a full reference GEMM. It is much cheaper than building a full $O(N^3)$ reference result.

Final Mental Model

INT8 tensor core GEMM is not just a matter of calling an MMA instruction. The tensor core is a fast arithmetic engine inside the SM, but it only performs well when the surrounding CUDA kernel delivers data efficiently.

The optimization path is:

Correct scalar idea → WMMA fragments → coalesced global memory → register reuse → shared memory reus

A good mental model is:

> The GEMM is a factory. Tensor cores are the machines. Registers are the workbench. Shared memory is the local warehouse. L2 and HBM are distant storage. Performance comes from arranging the factory so the machines never wait for parts.

The lecture's most important engineering lesson is that each optimization changes the bottleneck. Fixing global memory coalescing exposes register reuse. Improving register reuse exposes shared memory and occupancy trade-offs. Adding shared memory exposes synchronization, bank conflicts, and register spills. Preshuffling reduces repeated layout work but introduces preprocessing cost. The only reliable way to proceed is to combine first-principles reasoning with careful profiling after every change.